pair syntax in c++ and how to perform basic operations on it

In C++, a **pair** is a simple container from the Standard Library that holds **two values (possibly of different types)**. It lives in `<utility>` and is commonly used for things like coordinates, key–value data, or returning two values from a function.

---

1. Declaring a `pair`

cpp

```cpp
#include <utility>

std::pair<int, int> p;
```

Copy code

General syntax:

cpp

```cpp
std::pair<T1, T2> name;
```

Copy code

Example:

cpp

```cpp
std::pair<int, std::string> student;
```

Copy code

---

2. Initializing a `pair`

a) Direct initialization

cpp

Copy code

Copy code

```cpp
std::pair<int, int> p(10, 20);
```

b) Using `make_pair` (older but still common)

cpp

```cpp
auto p = std::make_pair(10, 20);
```

Copy code

c) Uniform initialization (modern C++)

cpp

```cpp
std::pair<int, int> p{10, 20};
```

Copy code

---

3. Accessing elements

A `pair` has two public members:

- `first`
- `second`

cpp

```cpp
std::pair<int, int> p{10, 20};

std::cout << p.first;   // 10
std::cout << p.second;  // 20
```

Copy code

---

4. Modifying elements

cpp

```cpp
p.first = 100;
p.second = 200;
```

Copy code

---

5. Comparing pairs

Pairs can be compared **lexicographically**:

1. `first` is compared
2. If `first` is equal, `second` is compared

cpp

```cpp
std::pair<int, int> p1{1, 2};
std::pair<int, int> p2{1, 3};

if (p1 < p2) {
    // true (because 2 < 3)
}
```

Copy code

Supported operators:

cpp

```cpp
==, !=, <, <=, >, >=
```

Copy code

ChatGPT Free offer ×

cpp

```cpp
std::pair<int, int> p1{5, 10};
std::pair<int, int> p2;

p2 = p1;  // copy assignment
```

Copy code

---

7. Swapping pairs

cpp

```cpp
std::pair<int, int> p1{1, 2};
std::pair<int, int> p2{3, 4};
```

Copy code

```
std::swap(p1, p2);
```

---

8. Using pairs with containers (very common)

a) Vector of pairs

cpp Copy code

```cpp
#include <vector>

std::vector<std::pair<int, int>> v;

v.push_back({1, 2});
v.push_back(std::make_pair(3, 4));
```

Accessing:

cpp Copy code

```cpp
for (auto &p : v) {
    std::cout << p.first << " " << p.second << "\n";
}
```

b) Map (key–value pairs)

cpp Copy code

```cpp
#include <map>

std::map<int, std::string> m;
m[1] = "one";

auto it = m.begin();
std::cout << it->first << " " << it->second;
```

---

9. Returning a pair from a function

cpp Copy code

```cpp
std::pair<int, int> getMinMax(int a, int b) {
    return {std::min(a, b), std::max(a, b)};
}
```

Usage:

cpp Copy code

```cpp
auto result = getMinMax(10, 5);
std::cout << result.first << " " << result.second;
```

---

10. Structured bindings (C++17+ 🔥)

This is the cleanest modern way to use pairs:

cpp Copy code

```cpp
std::pair<int, int> p{10, 20};

auto [x, y] = p;
std::cout << x << " " << y;
```

TL;DR

- `std::pair<T1, T2>` stores two values
- Access with `.first` and `.second`
- Supports comparison, assignment, swapping
- Heavily used in `map`, `vector`, and return values
- Use **structured bindings** in modern C++

If you want, I can also show **common pitfalls**, **interview questions**, or **how pairs compare to** `struct`.